

- shell 回收它所有的僵死子进程。如果任何作业因为收到一个未捕获的信号而终止，那么 shell 就输出一条消息到终端，消息中包含该作业的 PID 和对该信号的描述。

图 8-46 展示了一个 shell 会话示例。

```

linux> ./shell                                Run your shell program
>bogus
bogus: Command not found.                     Execve can't find executable
>foo 10
Job 5035 terminated by signal: Interrupt      User types Ctrl+C
>foo 100 &
[1] 5036 foo 100 &
>foo 200 &
[2] 5037 foo 200 &
>jobs
[1] 5036 Running      foo 100 &
[2] 5037 Running      foo 200 &
>fg %1
Job [1] 5036 stopped by signal: Stopped      User types Ctrl+Z
>jobs
[1] 5036 Stopped      foo 100 &
[2] 5037 Running      foo 200 &
>bg 5035
5035: No such process
>bg 5036
[1] 5036 foo 100 &
>/bin/kill 5036
Job 5036 terminated by signal: Terminated
> fg %2
Wait for fg job to finish
>quit
linux>                                         Back to the Unix shell

```

图 8-46 家庭作业 8.26 的 shell 会话示例

练习题答案

- 8.1 进程 A 和 B 是互相并发的，就像 B 和 C 一样，因为它们各自的执行是重叠的，也就是一个进程在另一个进程结束前开始。进程 A 和 C 不是并发的，因为它们的执行没有重叠；A 在 C 开始之前就结束了。
- 8.2 在图 8-15 的示例程序中，父子进程执行无关的指令集合。然而，在这个程序中，父子进程执行的指令集合是相关的，这是有可能的，因为父子进程有相同的代码段。这会是一个概念上的障碍，所以请确认你理解了本题的答案。图 8-47 给出了进程图。
- A. 这里的关键点是子进程执行了两个 printf 语句。在 fork 返回之后，它执行第 6 行的 printf，然后它从 if 语句中出来，执行第 7 行的 printf 语句。下面是子进程产生的输出：

```

p1: x=2
p2: x=1

```

- B. 父进程只执行第 7 行的 printf:

```

p2: x=0

```

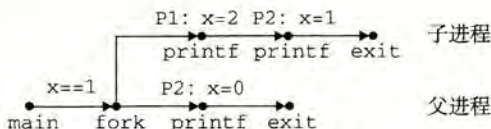


图 8-47 练习题 8.2 的进程图

- 8.3 我们知道序列 *acbc*、*abcc* 和 *bacc* 是可能的，因为它们对应有进程图的拓扑排序(图 8-48)。而像 *bcac* 和 *cbca* 这样的序列不对应有任何拓扑排序，因此它们是不可行的。

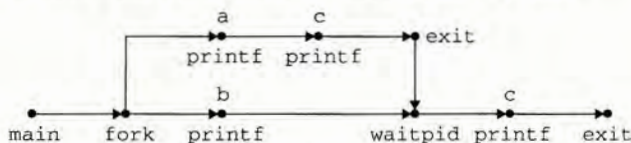


图 8-48 练习题 8.3 的进程图

- 8.4 A. 只简单地计算进程图(图 8-49)中 `printf` 顶点的个数就能确定输出行数。在这里，有 6 个这样的顶点，因此程序会打印 6 行输出。
- B. 任何对应有进程图的拓扑排序的输出序列都是可能的。例如：*Hello*、*1*、*0*、*Bye*、*2*、*Bye* 是可能的。

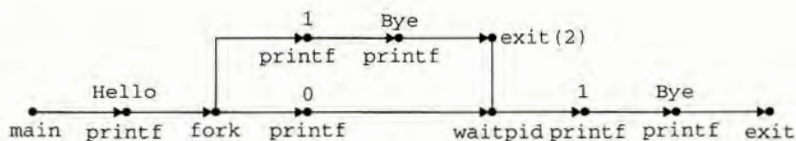


图 8-49 练习题 8.4 的进程图

- 8.5 code/ecf/snooze.c

```
1 unsigned int snooze(unsigned int secs) {
2     unsigned int rc = sleep(secs);
3
4     printf("Slept for %d of %d secs.\n", secs-rc, secs);
5     return rc;
6 }
```

code/ecf/snooze.c

- 8.6 code/ecf/myecho.c

```
1 #include "csapp.h"
2
3 int main(int argc, char *argv[], char *envp[])
4 {
5     int i;
6
7     printf("Command-line arguments:\n");
8     for (i=0; argv[i] != NULL; i++)
9         printf("    argv[%2d]: %s\n", i, argv[i]);
10
11     printf("\n");
12     printf("Environment variables:\n");
13     for (i=0; envp[i] != NULL; i++)
14         printf("    envp[%2d]: %s\n", i, envp[i]);
15
16     exit(0);
17 }
```

code/ecf/myecho.c

- 8.7 只要休眠进程收到一个未被忽略的信号，`sleep` 函数就会提前返回。但是，因为收到一个 `SIGINT` 信号的默认行为就是终止进程(图 8-26)，我们必须设置一个 `SIGINT` 处理程序来允许 `sleep` 函数返回。处理程序简单地捕获 `SIGNAL`，并将控制返回给 `sleep` 函数，该函数会立即返回。

code/ecf/snooze.c

```
1 #include "csapp.h"
2
```

```

3  /* SIGINT handler */
4  void handler(int sig)
5  {
6      return; /* Catch the signal and return */
7  }
8
9  unsigned int snooze(unsigned int secs) {
10     unsigned int rc = sleep(secs);
11
12     printf("Slept for %d of %d secs.\n", secs-rc, secs);
13     return rc;
14 }
15
16 int main(int argc, char **argv) {
17
18     if (argc != 2) {
19         fprintf(stderr, "usage: %s <secs>\n", argv[0]);
20         exit(0);
21     }
22
23     if (signal(SIGINT, handler) == SIG_ERR) /* Install SIGINT */
24         unix_error("signal error\n");      /* handler      */
25     (void)snooze(atoi(argv[1]));
26     exit(0);
27 }

```

code/ecf/snooze.c

- 8.8 这个程序打印字符串“213”，这是卡内基-梅隆大学 CS: APP 课程的缩写名。父进程开始时打印“2”，然后创建子进程，子进程会陷入一个无限循环。然后父进程向子进程发送一个信号，并等待它终止。子进程捕获这个信号(中断这个无限循环)，对计数器值(从初始值 2)减一，打印“1”，然后终止。在父进程回收子进程之后，它对计数器值(从初始值 2)加一，打印“3”，并且终止。